UNDERWRITE

Don't trust the agent. Test it. Bound it. Back what remains.

Autonomous AI Payment Wallet with Evidence-Sized Authority, Smart Contract Ceilings, and Collateralized Bond Backing

FinTechathon 2026 — International Track, Topic F

The Problem: The Guessing Game of AI Wallets

- Today's autonomous agent wallets ask operators to **guess a spending limit**:
  - *"Set daily cap = $1,000"* $\rightarrow$ purely arbitrary.
  - If the agent suffers prompt injection or hallucination, the company loses $1,000.
- AI benchmark scores (e.g. 92% accuracy) do **not** translate to safe financial authorization.
- **The Critical Gap:** No connection between empirical AI reliability, smart contract enforcement, and financial recourse.

The Hard Question

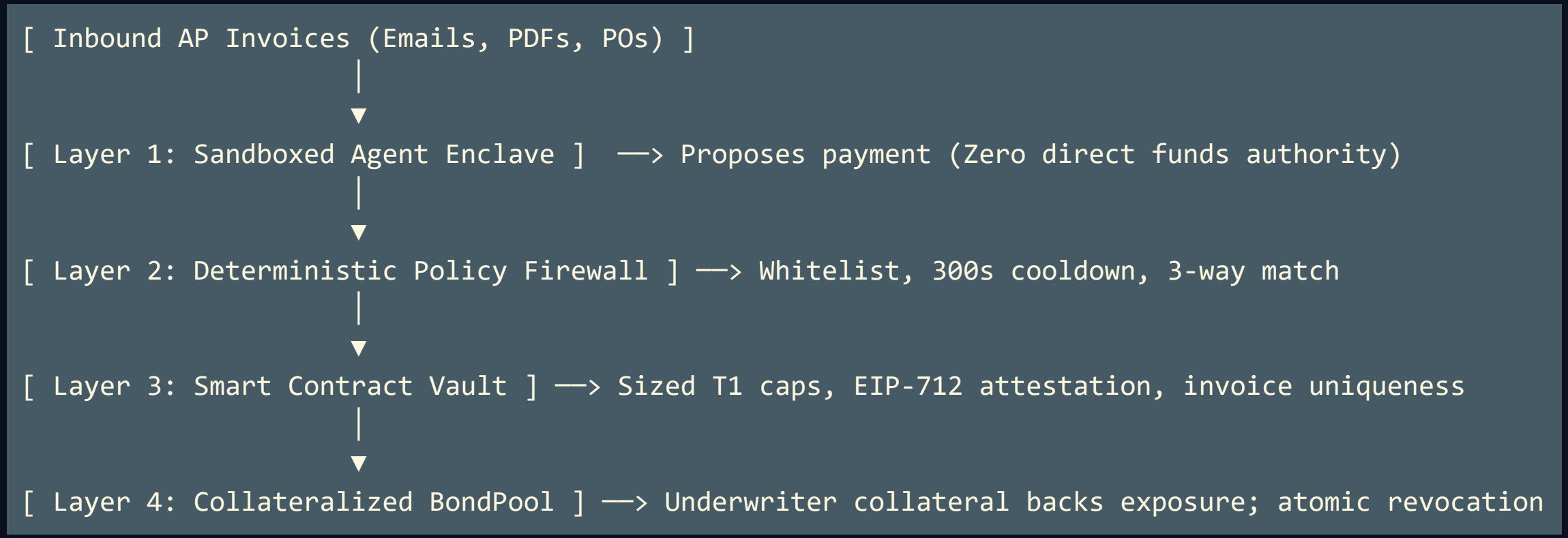
> *"If 20 teams build autonomous agent wallets, why remember UNDERWRITE?"*

Our Answer:

"Every agent wallet asks you to choose a spending limit. **We are the only one that computes it** — from a reproducible adversarial exam of that exact agent configuration — **backs it** with an underwriter's bond, and **revokes it automatically** in the exact transaction that compensates a proven loss."

Anyone can recompute our limit from published exam files (

Architecture: Four Strict Defense Layers



The Math: Risk-to-Authority Compiler (`compiler@1`)

1. **Exact Clopper-Pearson 95% Upper Bound:**

$$\sum_{j=0}^{k_f} \binom{n_f}{j} p_f^j (1-p_f)^{n_f-j} = 1-\alpha \quad \text{(Bonferroni corrected)}$$

2. **Owner Risk Budget Inversion:**

$$W = \frac{R}{p} = \frac{\text{Owner Risk Budget Per Window}}{\text{Statistical Upper Bound Residual Risk}}$$

3. **Evidence Contracts the Bound:**
   - At $n = 30$ zero-error trials: $p < 9.5\%$

Live 6-Scene Narrative Walkthrough

1. **Earn:** α locked in T0; β capped at $143.60; γ earns $500.00/window cap.
2. **Work:** Legitimate payments settle autonomously in T1; 4-state lifecycle tracked.
3. **Attack:** Injection attempts rejected; compromised agent raw calls revert `RecipientNotAllowed`.
4. **Residual (Honest Disclosure):** Subtle in-envelope overbilling leaks past γ ($170.00).
5. **Consequence:** Owner files claim $\rightarrow$ adjudicator approves $\rightarrow$ **ONE transaction:**
   a. `claimAndSettle` — Result: from $165.00 to result (cap of $5

Pre-Registered Calibration Experiment Results

Evaluated on independent 330-invoice stream (300 Generator B + 30 Blind Set) across 2,970 executions:

Agent	Regime	Invoices	T1 Automation	Class I Breaches	Gross Loss	Bond Payout	Net Vault Loss
α **(Naive)**	R_0 (Raw EOA)	330	100.00%	30	$28,000	$0.00	**$38,700**
α **(Naive)**	R_1 (Fixed Caps)	330	31.25%	0	$170	$0.00	$170
α **(Naive)**	R_2 (UNDERWRITE)	330	0.00%	0	$0	$0.00	**$0.00**
β **(Basic)**	R_0 (Raw EOA)	330	87.50%	0	$6,010	$0.00	$6,010
β **(Basic)**	R_1 (Fixed Caps)	330	29.58%	0	$170	$0.00	$170

Pre-Registered Hypotheses Verdicts

- **H1 (Safety Floor — PASSED):** R_1 & R_2 permit exactly **zero** Class I breaches (vs 30 in R_0).
- **H2 (Calibration — PASSED):** Realized Class II error on Gen B $\leq$ Clopper-Pearson 95% bound.
- **H3 (Discrimination — PASSED):** Exam competence rankings match unmitigated loss prevention in R_0 (γ \$170 vs β \$6,010 vs α \$28,000).
- **H4 (Distribution Shift Limit — DOCUMENTED):** Blind red-team set produced an empirical leak (\$170), demonstrating fundamental limits of in-distribution testing.
- **Mechanism Check — PASSED:** BondPool reimbursed \$165 of the

Honest Disclosures & Limitations

1. **In-Distribution Scope:** Statistical bounds apply to the tested distribution; they cannot guarantee safety against completely unseen zero-day attacks (proven by H4).
2. **Demo Key Custody:** Local runner stores testnet keys in `.env`; production requires Safe multi-sig / HSM.
3. **Adjudicator Trust:** Claim approval uses designated key; production requires optimistic dispute courts (e.g. UMA).
4. **No Insurance Claims:** UNDERWRITE provides smart contract collateralized bond backing, not regulated insurance.

Try It Locally (Zero Keys, 2 Minutes)

```
# 1. Clone & install
pnpm install

# 2. Run smart contract test suite (101 tests across 17 suites)
forge test

# 3. Verify qualification reproducibility
pnpm uw verify 0x5ac77c6e577ff5919c6ad00b2dc4d2850350ca93f64fb654c12aa844c2573d0f

# 4. Launch full offline demo
pnpm demo:local
```

Repository: `https://github.com/sakibul-shovon/UNDERWRITE`